

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.
(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I D.AnandPaul(192425207) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network condition

1. Network Throughput and Packet Loss Calculation

Aim

To calculate network throughput and packet loss percentage, classify the network as efficient or inefficient, and display a complete performance summary.

Given Values

- Total data received = 8000 bits
- Time = 4 seconds
- Packets sent = 200
- Packets received = 180

PSEUDOCODE:

BEGIN

dataReceived ← 8000

time ← 4

packetsSent ← 200

packetsReceived ← 180

IF time = 0 THEN

 DISPLAY "Error: Time cannot be zero"

 STOP

END IF

IF packetsSent = 0 THEN

 DISPLAY "Error: Total packets sent cannot be zero"

 STOP

END IF

throughput $\leftarrow$ dataReceived / time

lostPackets $\leftarrow$ packetsSent - packetsReceived

packetLoss $\leftarrow$ (lostPackets / packetsSent) $\times$ 100

DISPLAY "Network Throughput = ", throughput, " bps"

DISPLAY "Packet Loss = ", packetLoss, "%"

IF throughput > 1500 AND packetLoss < 10 THEN

 status $\leftarrow$ "Efficient"

ELSE

 status $\leftarrow$ "Inefficient"

END IF

DISPLAY "Network Performance Summary"

DISPLAY "Data Received = ", dataReceived, " bits"

DISPLAY "Time = ", time, " seconds"

DISPLAY "Packets Sent = ", packetsSent

DISPLAY "Packets Received = ", packetsReceived

DISPLAY "Lost Packets = ", lostPackets

DISPLAY "Throughput = ", throughput, " bps"

DISPLAY "Packet Loss = ", packetLoss, "%"

DISPLAY "Network Status = ", status

END

Result

Network Status = Inefficient

2. Bandwidth Utilization Monitoring

Aim

To continuously monitor bandwidth utilization of multiple network links, identify overloaded links, and recommend switching traffic to backup links.

Pseudocode

BEGIN

links ← ["Link1", "Link2", "Link3", "Link4"]
utilization ← [0, 0, 0, 0]

WHILE TRUE DO

 DISPLAY "Starting bandwidth monitoring"

 FOR i ← 0 TO length(links) - 1 DO

 utilization[i] ← GET_BANDWIDTH_USAGE(links[i])

 DISPLAY links[i], " Utilization = ", utilization[i], "%"

 IF utilization[i] > 80 THEN

 DISPLAY "WARNING: ", links[i], " is overloaded"

 DISPLAY "Recommendation: Switch traffic to backup link"

 SWITCH_TRAFFIC_TO_BACKUP(links[i])

 ELSE

 DISPLAY links[i], " is operating normally"

 END IF

 END FOR

 WAIT 10 seconds

END WHILE

END

Explanation

- The algorithm stores each link's bandwidth utilization in an **array**.
- A **FOR** loop checks every network link.
- If utilization is **greater than 80%**, the link is considered overloaded.
- Traffic is recommended to be moved to a **backup link**.
- The **WHILE** loop continuously monitors the network.
- The algorithm waits for a fixed interval before collecting new values.

Result

This algorithm helps prevent **network congestion**, balances bandwidth usage, and improves overall network performance by shifting traffic away from overloaded links.

3. Best Path Selection Based on Link Quality

Aim

To continuously monitor bandwidth, delay, and utilization and select the best communication path based on overall link quality.

Pseudocode

BEGIN

paths ← ["Path1", "Path2", "Path3", "Path4", "Path5"]

bandwidth ← [0, 0, 0, 0, 0]

delay ← [0, 0, 0, 0, 0]

utilization ← [0, 0, 0, 0, 0]

qualityScore ← [0, 0, 0, 0, 0]

WHILE TRUE DO

 highestScore ← -1

 bestPath ← "None"

 FOR i ← 0 TO length(paths) - 1 DO

 bandwidth[i] ← GET_BANDWIDTH(paths[i])

 delay[i] ← GET_DELAY(paths[i])

 utilization[i] ← GET_UTILIZATION(paths[i])

 IF bandwidth[i] <= 0 OR paths[i] = FAILED THEN

```
    DISPLAY "ALERT: ", paths[i], " is failed"  
    CONTINUE
```

```
END IF
```

```
IF utilization[i] > 80 THEN
```

```
    DISPLAY "ALERT: ", paths[i], " is overloaded"  
    CONTINUE
```

```
END IF
```

```
qualityScore[i] ←  
    (0.5 × bandwidth[i])  
    - (0.3 × delay[i])  
    - (0.2 × utilization[i])
```

```
DISPLAY paths[i], " Quality Score = ", qualityScore[i]
```

```
IF qualityScore[i] > highestScore THEN  
    highestScore ← qualityScore[i]  
    bestPath ← paths[i]  
END IF
```

```
END FOR
```

```
IF bestPath = "None" THEN  
    DISPLAY "ALERT: No suitable path available"  
ELSE  
    DISPLAY "Best Communication Path = ", bestPath  
    DISPLAY "Highest Quality Score = ", highestScore  
    ROUTE_TRAFFIC(bestPath)  
END IF
```

```
WAIT 10 seconds
```

```
END WHILE
```

```
END
```

Explanation of Quality Score

The algorithm considers three important factors:

- **Higher bandwidth** → improves the score.
- **Lower delay** → improves the score.
- **Lower utilization** → improves the score.

The formula is:

$$\text{Quality Score} = (0.5 \times \text{Bandwidth}) - (0.3 \times \text{Delay}) - (0.2 \times \text{Utilization})$$

The path with the **highest score** is selected.

Result

The algorithm continuously adapts to changing network conditions. It avoids overloaded or failed links, automatically selects an alternate path, and generates alerts. Therefore, it improves **routing efficiency, reliability, fault tolerance, and network performance**.